

PASO A PASO COMPLETO PARA D

AUTOMÁTICO CON CLAUDE:

para D

Ahora puedes lanzarlo para 500000 eventos con confianza:

(bash)

```
tmux new -s sim_D13
```

```
bash auto_sim.sh 13.0 500000 25000
```

Y para desconectarte sin matar el proceso: **Ctrl+B** luego **D**. Para reconectarte más tarde:
`tmux attach -t sim_D13`.

Cuando quieras hacer otra D, abre una nueva sesión tmux con otro nombre:

(bash)

```
tmux new -s sim_D20
```

```
bash auto_sim.sh 20.0 500000 25000
```

para theta

```
tmux new -s sim_ang30
```

```
bash auto_sim_angulo.sh 30 500000 25000
```

para D y theta

D=7

```
tmux new -s D7_ang0; bash auto_sim_D_angulo.sh 7.0 0 500000 25000
```

```
tmux new -s D7_ang20; bash auto_sim_D_angulo.sh 7.0 20 500000 25000
```

```
tmux new -s D7_ang40; bash auto_sim_D_angulo.sh 7.0 40 500000 25000
```

```
tmux new -s D7_ang60; bash auto_sim_D_angulo.sh 7.0 60 500000 25000
```

D=13

```
tmux new -s D13_ang0; bash auto_sim_D_angulo.sh 13.0 0 500000 25000
```

... etc

```
tmux new -s nombre # crear sesión nueva
```

Ctrl+B luego D # desconectar sin matar (la sim sigue corriendo)

```
tmux attach -t nombre # reconectarse a una sesión
```

```
tmux ls # listar todas las sesiones activas
```

```
tmux kill-session -t nombre # matar una sesión cuando ha terminado
```

DEEPSEEK:

Parte 1: Cambiar la geometría en el servidor (sultan)

1. Conéctese a sultan

(bash)

```
ssh tfg_2025_pablo@sultan.unizar.es
```

(Use su contraseña)

2. Vaya a la carpeta del código fuente

(bash)

```
cd ~/G4TomoDet_TFG2025/src
```

3. Edite el archivo de geometría

(bash)

```
nano DetectorConstruction.cc
```

4. Busque las líneas que controlan las distancias (aproximadamente línea 70-80)

```
fGapBigChamber = 13. *cm;    // gap between chambers pairs
```

Cámbielas a los valores que quiera.

5. Guarde y salga de nano

- Ctrl + O → Enter
- Ctrl + X

6. Vaya a la carpeta build y recompila

(bash)

```
cd ~/G4TomoDet_TFG2025/build
```

```
make -j18
```

Espere a que termine (verá algo como [100%] Built target G4TomoDet).

Parte 2: Preparar el macro de simulación

7. Edite el macro muons.mac

(bash)

```
nano ~/G4TomoDet_TFG2025/macros/muons.mac
```

8. Asegúrese de que el macro tenga el siguiente contenido (cópielo si es necesario)

(bash)

/vis/disable (esto hay q añadirlo pq sino da violación de segmento)

```
/run/initialize
```

```
/TomoDet/setVerbose 0
```

```
/tracking/verbose 0
```

```
/gun/particle mu-
```

```
/gen/option 3
```

```
/gen/file ../macros/toyfile_model.root
```

```
/TomoDet/analyse/setFileName ../../Prueba_D9.root
```

```
/event/printModulo 10
```

```
/random/setSeeds 46531 63205
```

```
/random/setSavingFlag 0
```

```
/random/saveEachEventFlag 0
```

```
/run/beamOn 100
```

(El número 100 es el número de eventos. Cámbielo si quiere más y G4TomoDet_MiSimulacion tm bel que yo quiera)

9. Guarde y salga

- Ctrl + O → Enter → Ctrl + X

Parte 3: Ejecutar la simulación

10. Lance la simulación (desde build)

```
(bash)
```

```
cd ~/G4TomoDet_TFG2025/build
```

```
./G4TomoDet ../macros/muons.mac > simulacion_D9.log
```

Espere a que termine (10-30 segundos para 100 eventos). Verá el prompt de nuevo cuando termine. (Aun asi da violación de segmento pero creo q es solo q no visualiza y ya)

11. Compruebe que se ha generado el archivo .root

```
(bash)
```

```
ls -la ~/G4TomoDet_Prueba_D9.root
```

Si el archivo existe y tiene fecha actual → **bien**. Esta en tfg_2025_pablo@sultan:~\$ (cd)

Si no, mire el log:

```
bash
```

```
cat simulacion.log | tail -30
```

Parte 4: Analizar los datos y generar el .csv

12. Copie el script de análisis

```
(bash)
```

```
cd ~/scripts
```

```
cp extraer_datos.C extraer_MIO.C
```

13. Edite el script para que lea su archivo y guarde con otro nombre

```
(bash)
```

```
nano extraer_MIO.C
```

Cambie estas 4 líneas:

Línea 0:

```
void extraer_MIO() {          ---          en vez de extraer_datos(){
```

Línea 1 (el archivo de entrada):

```
cpp
```

```
TFile *file = TFile::Open("../G4TomoDet_Prueba_D9.root"); (sin /data/...)
```

Línea 2 (el archivo de salida):

```
cpp
```

```
ofstream out("Prueba_D9.csv");
```

Línea 3 (el mensaje final):

```
cpp
```

```
cout << "Datos guardados en Prueba_D9.csv" << endl;
```

14. Guarde y salga

- Ctrl + O → Enter → Ctrl + X

15. Ejecute el análisis

```
(bash)
```

```
root -l -q extraer_MIO.C
```

Debería ver el mensaje: Datos guardados en coordenadas_MIO.csv (Prueba_D9.csv)

16. Compruebe que el .csv se ha creado

(bash)

ls -la coordenadas_MIO.csv

Parte 5: Traer el .csv a su ordenador (Windows)

Ahora abra UNA NUEVA terminal en su ordenador (WSL local, NO conectado a sultan).

Verá un prompt como pablo@hpdepablocc:~\$.

17. Copie el archivo de sultan a su WSL local

(bash)

scp tfg_2025_pablo@sultan.unizar.es:~/scripts/Prueba_D9.csv ~/

(Use su contraseña de sultan)

18. Verifique que el archivo está en su WSL

bash

ls -la ~/Prueba_D9.csv

19. Vea el nombre de su escritorio en Windows

pablo@hpdepablocc:~\$ cp ~/Prueba_D9.csv /mnt/c/Users/pablo/Downloads

(TAL CUAL DEEPSKEEK)

Parte 1: Cambiar la geometría en el servidor (sultan)

1. Conéctese a sultan

bash

ssh tfg_2025_pablo@sultan.unizar.es

(Use su contraseña)

2. Vaya a la carpeta del código fuente

bash

cd ~/G4TomoDet_TFG2025/src

3. Edite el archivo de geometría

bash

nano DetectorConstruction.cc

4. Busque las líneas que controlan las distancias (aproximadamente línea 70-80)

Dentro de nano, use Ctrl + W y busque fGapBigChamber.

Verá algo como:

cpp

fGapBigChamber = 13. *cm; // gap between chambers pairs

fGapSmallChamber = 7.0 *cm; // gap between chambers of a pair

Cámbielas a los valores que quiera.

Ejemplo (distancia grande = 15 cm, distancia pequeña = 5 cm):

cpp

fGapBigChamber = 15. *cm; // gap between chambers pairs

fGapSmallChamber = 5.0 *cm; // gap between chambers of a pair

Importante: La distancia grande debe ser mayor que la pequeña (como ha dicho usted). Si es al revés, el programa crashea.

5. Guarde y salga de nano

- **Ctrl + O → Enter**
- **Ctrl + X**

6. Vaya a la carpeta build y recompila

bash

cd ~/G4TomoDet_TFG2025/build

rm CMakeCache.txt

```
cmake .. -DCMAKE_MODULE_PATH=$ROOTSYS/etc/cmake
```

```
make -j8
```

Espere a que termine (verá algo como [100%] Built target G4TomoDet).

Parte 2: Preparar el macro de simulación

7. Edite el macro muons.mac

```
bash
```

```
nano ~/G4TomoDet_TFG2025/macros/muons.mac
```

8. Asegúrese de que el macro tenga el siguiente contenido (cópelo si es necesario)

```
bash
```

```
/vis/disable
```

```
/run/initialize
```

```
/TomoDet/setVerbose 0
```

```
/tracking/verbose 0
```

```
/gun/particle mu-
```

```
/gen/option 3
```

```
/gen/file ../macros/toyfile_model.root
```

```
/TomoDet/analyse/setFileName ../../G4TomoDet_MiSimulacion.root
```

```
/event/printModulo 10
```

```
/random/setSeeds 46531 63205
```

```
/random/setSavingFlag 0
```

```
/random/saveEachEventFlag 0
```

```
/run/beamOn 100
```

(El número 100 es el número de eventos. Cámbielo si quiere más).

9. Guarde y salga

- Ctrl + O → Enter → Ctrl + X
-

Parte 3: Ejecutar la simulación

10. Lance la simulación (desde build)

bash

cd ~/G4TomoDet_TFG2025/build

./G4TomoDet ../macros/muons.mac > simulacion.log

Espere a que termine (10-30 segundos para 100 eventos). Verá el prompt de nuevo cuando termine.

11. Compruebe que se ha generado el archivo .root

bash

ls -la ~/G4TomoDet_MiSimulacion.root

Si el archivo existe y tiene fecha actual → bien.

Si no, mire el log:

bash

cat simulacion.log | tail -30

Parte 4: Analizar los datos y generar el .csv

12. Copie el script de análisis

bash

cd ~/scripts

cp extraer_datos.C extraer_MIO.C

13. Edite el script para que lea su archivo y guarde con otro nombre

bash

nano extraer_MIO.C

Cambie estas 3 líneas:

Línea 1 (el archivo de entrada):

cpp

TFile *file = TFile::Open("../G4TomoDet_MiSimulacion.root");

Línea 2 (el archivo de salida):

cpp

ofstream out("coordenadas_MIO.csv");

Línea 3 (el mensaje final):

cpp

```
cout << "Datos guardados en coordenadas_MIO.csv" << endl;
```

14. Guarde y salga

- Ctrl + O → Enter → Ctrl + X

15. Ejecute el análisis

bash

```
root -l -q extraer_MIO.C
```

Debería ver el mensaje: Datos guardados en coordenadas_MIO.csv

16. Compruebe que el .csv se ha creado

bash

```
ls -la coordenadas_MIO.csv
```

Parte 5: Traer el .csv a su ordenador (Windows)

Ahora abra UNA NUEVA terminal en su ordenador (WSL local, NO conectado a sultan).

Verá un prompt como pablo@hpdepablocc:~\$.

17. Copie el archivo de sultan a su WSL local

bash

```
scp tfg_2025_pablo@sultan.unizar.es:~/scripts/coordenadas_MIO.csv ~/
```

(Use su contraseña de sultan)

18. Verifique que el archivo está en su WSL

bash

```
ls -la ~/coordenadas_MIO.csv
```

19. Vea el nombre de su escritorio en Windows

bash

```
ls /mnt/c/Users/pablo/
```

Si ve Desktop (inglés) o Escritorio (español), use el que corresponda.

20. Copie el archivo a su escritorio de Windows

Si su escritorio se llama Desktop:

bash

```
cp ~/coordenadas_MIO.csv /mnt/c/Users/pablo/Desktop/
```

Si su escritorio se llama Escritorio:

bash

cp ~/coordenadas_MIO.csv "/mnt/c/Users/pablo/Escritorio/"

21. Abra el archivo desde Windows

Vaya a su escritorio y haga doble clic en coordenadas_MIO.csv. Se abrirá con Excel o el programa que tenga asignado.

Resumen de comandos (para copiar y pegar en orden):

En sultan:

bash

cd ~/G4TomoDet_TFG2025/src

nano DetectorConstruction.cc *# cambiar fGapBigChamber y fGapSmallChamber*

cd ~/G4TomoDet_TFG2025/build

make -j8

./G4TomoDet ../macros/muons.mac > simulacion.log

cd ~/scripts

cp extraer_datos.C extraer_MIO.C

nano extraer_MIO.C *# cambiar las 3 líneas*

root -l -q extraer_MIO.C

En su WSL local (nueva terminal):

bash

scp tfg_2025_pablo@sultan.unizar.es:~/scripts/coordenadas_MIO.csv ~/

cp ~/coordenadas_MIO.csv /mnt/c/Users/pablo/Desktop/

PARA CAMBIOS DE THETA

Resumen de lo que ha dicho (y es correcto):

Si quiere...	Hace esto...	El resto del proceso (compilar, simular, analizar)
Cambiar el rango de theta (ej. de 0-60° a 0-30°)	Modificar thetamax en PrimaryGeneratorAction.cc (opción 3)	Igual (compilar, lanzar, extraer datos)
Usar otro modelo (ej. Reyna, Bogdanova, Shukla)	Cambiar /gen/option en muons.mac (poner 4, 5 o 6)	Igual (no necesita tocar código, solo el macro)
Usar un theta fijo (no aleatorio)	Modificar el código para asignar theta = valor	Igual (pero más trabajo, no recomendado)

Lo que NO cambia (siempre es igual):

1. Compilar (si tocó código .cc):

bash

cd ~/G4TomoDet_TFG2025/build

make -j18

2. Lanzar la simulación:

bash

./G4TomoDet ../macros/muons.mac > simulacion.log

3. Analizar los datos (con su script extraer_MIO.C adaptado a su archivo .root):

bash

cd ~/scripts

root -l -q extraer_MIO.C

4. Traer el .csv a Windows (con scp y cp).
-

Tabla de opciones según lo que quiera simular:

Escenario	Qué cambia	Dónde
Theta de 0-30°	thetamax = 30.	PrimaryGeneratorAction.cc (línea ~380)
Theta de 0-80°	thetamax = 80.	PrimaryGeneratorAction.cc
Modelo Reyna	/gen/option 4	muons.mac
Modelo Bogdanova	/gen/option 5	muons.mac
Modelo Shukla	/gen/option 6	muons.mac
Geometría (distancias D)	fGapBigChamber, fGapSmallChamber	DetectorConstruction.cc
Rotar detectores	fchamberRotAngle X/Y/Z	DetectorConstruction.cc

Ejemplo práctico:

Caso A: Quiere simular muones más verticales (0-30°) con modelo Guan

1. Edite PrimaryGeneratorAction.cc:

cpp

```
thetamax = 30.; // antes era 60
```

2. Compile:

bash

```
cd ~/G4TomoDet_TFG2025/build
```

```
make -j18
```

3. Asegúrese de que muons.mac tiene /gen/option 3
4. Lance la simulación (pasos 10-11 del paso a paso)
5. Analice igual (su script extraer_MIO.C)

Caso B: Quiere probar el modelo Shukla (sin tocar código)

1. Edite muons.mac:

bash

/gen/option 6 # antes era 3

- 2. No necesita compilar (el código ya tiene implementada la opción 6)**
- 3. Lance la simulación**
- 4. Analice igual**

Conclusión:

Pregunta	Respuesta
¿Cambiar theta = cambiar thetamax?	Sí (y recompilar)
¿Cambiar modelo = cambiar /gen/option?	Sí (no recompilar)
¿El resto (simular, analizar, guardar) es igual?	Sí (exactamente los mismos comandos)

Soluciones (orden de mejor a peor):

Opción 1 (recomendada): Hacer varias simulaciones de 100k y unir las

Para su nueva geometría (D cambiado), haga **5 simulaciones de 100k eventos cada una**, con **distinta semilla aleatoria**.

En muons.mac, cambie la semilla en cada simulación:

Simulación	Semilla (dos números)
1	/random/setSeeds 46531 63205
2	/random/setSeeds 12345 67890
3	/random/setSeeds 98765 43210
4	/random/setSeeds 11111 22222
5	/random/setSeeds 33333 44444

Luego una los 5 archivos .csv en uno solo de 500k eventos.

Unir .csv en Linux:

bash

```
cat coordenadas_sim1.csv > coordenadas_total.csv
```

```
tail -n +2 coordenadas_sim2.csv >> coordenadas_total.csv
```

```
tail -n +2 coordenadas_sim3.csv >> coordenadas_total.csv
```

```
tail -n +2 coordenadas_sim4.csv >> coordenadas_total.csv
```

```
tail -n +2 coordenadas_sim5.csv >> coordenadas_total.csv
```

(El tail -n +2 omite la cabecera a partir del segundo archivo).

Opción 2 (más sencilla): Submuestrear los 500k a 100k

Si ya tiene 500k para la geometría original, **tome solo los primeros 100k** de ese archivo para comparar con los 100k de la nueva geometría.

Desde el .csv:

bash

```
head -100001 muon_hits_complete.csv > muon_hits_100k.csv
```

(Las 100.001 primeras líneas = cabecera + 100k eventos).

Desde el .root (más preciso): En ROOT, puede leer solo las primeras N entradas del árbol. Pero es más complicado.

Opción 3 (si el tiempo apremia): Acepte la diferencia y sea honesto en su TFG

En su informe, puede decir claramente:

"Para la geometría original se dispone de 500k eventos, mientras que para las geometrías modificadas se simularon 100k eventos debido a limitaciones de tiempo/cómputo. Las barras de error reflejan la diferencia de estadística."

No es ideal, pero es **honesto y defendible**.